

Laporan Tugas Besar 2

IF2010 Pemrograman Berorientasi Objek 2025/2026

Banana Republic

Kode Kelompok : BCC

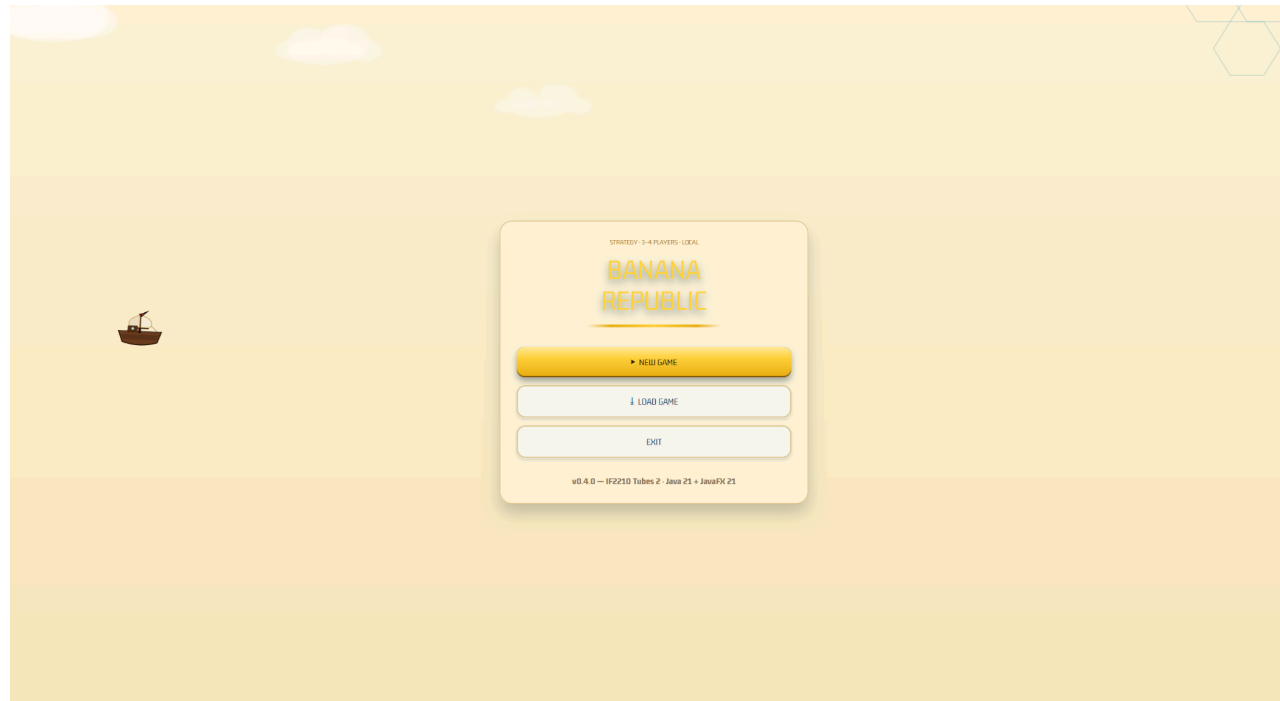
Nama Kelompok : BurntCheeseCake

1. 13524026 Made Branenda Jordhy
2. 13524028 Muhammad Nur Majiid
3. 13524034 Jason Edward Salim
4. 13524067 Bryan Pratama Putra Hendra
5. 13524068 Athilla Zaidan Zidna Fann



BCC - Burnt Cheese Cake

1. Deskripsi Umum Aplikasi



Gambar 1.1. Menu utama Banana Republic

Banana Republic adalah aplikasi desktop berbasis Java yang menghadirkan pengalaman bermain board game strategi secara lokal untuk tiga hingga empat pemain dalam satu mesin. Aplikasi ini merupakan karya orisinal yang terinspirasi oleh mekanisme klasik The Settlers of Catan, namun dibungkus dalam tema naratif yang unik bertajuk "Banana Republic" yang kaya akan nuansa tropis dan politik. Inti permainannya berputar pada ekosistem ekonomi sumber daya yang terdiri dari lima jenis material strategis, yaitu Kayu, Bata, Gandum, Bijih, dan Pisang, yang masing-masing dihasilkan oleh jenis wilayah tertentu di atas papan permainan berbentuk hexagonal. Pemain secara bergilir melempar dua buah dadu untuk menentukan angka produksi, lalu mengumpulkan sumber daya dari wilayah-wilayah yang mereka kuasai guna

membangun infrastruktur. Infrastruktur yang dapat didirikan meliputi Monitoring Post (pos pemantauan yang berfungsi sebagai basis permukiman), Pipe (pipa yang menghubungkan pos-pos untuk membentuk jaringan transportasi), serta Laboratory (laboratorium yang merupakan upgrade dari Monitoring Post dan memberikan produksi ganda serta poin kemenangan tambahan). Selain aspek konstruksi, permainan ini juga menawarkan mekanisme perdagangan yang mendalam, baik secara domestik antar-pemain maupun perdagangan maritim melalui pelabuhan-pelabuhan yang tersebar di papan, dengan rasio pertukaran yang bervariasi mulai dari dua banding satu untuk sumber daya spesifik hingga empat banding satu untuk pertukaran generik. Seluruh interaksi permainan dikemas dalam antarmuka grafis yang responsif dan menarik secara visual, dengan papan hexagonal yang dapat di-zoom, di-pan, dan di-drag menggunakan mouse, disertai animasi dadu yang halus, efek suara pada setiap aksi, serta latar belakang animasi yang memberikan nuansa hidup pada tiap layar.



Gambar 1.2. Tampilan di dalam game Banana Republic

Aplikasi ini telah berkembang menjadi produk yang sangat komprehensif dengan fitur-fitur pendukung yang memadukan aspek teknis dan pengalaman pengguna secara harmonis. Salah satu fitur unggulan adalah sistem audio terintegrasi yang menyediakan background musik latar (BGM) yang berbeda untuk menu utama dan sesi permainan, yang bahkan berotasi otomatis antar dua trek musik selama gameplay, serta delapan kategori efek suara (SFX) yang merespons setiap interaksi mulai dari klik tombol, pembangunan infrastruktur, pelemparan dadu, konfirmasi perdagangan, hingga pergerakan Nimon Ungu. Aplikasi juga dilengkapi dengan sistem persistensi save dan load game yang mampu menyimpan seluruh keadaan permainan ke dalam format JSON maupun serialisasi Java (.ser), sehingga pemain dapat melanjutkan permainannya kapan saja. Terdapat pula mekanisme Nimon Ungu yang aktif ketika angka tujuh muncul dari lemparan dadu, mewajibkan pemain dengan lebih dari tujuh kartu sumber daya untuk membuang setengah dari kekayaannya, memindahkan penanda Nimon ke tile baru, dan mencuri satu sumber daya secara acak dari pemain lain yang berbatasan dengan tile tersebut. Untuk menjaga kelancaran permainan, sistem turn timer diimplementasikan dengan hitungan mundur selama fase perdagangan dan pembangunan, yang secara otomatis mengakhiri giliran pemain ketika waktu habis dan memberikan peringatan visual berwarna merah saat tersisa kurang dari sepuluh detik. Sebelum pertandingan dimulai, aplikasi juga menjalankan fase penentuan urutan awal di mana setiap pemain melempar dadu secara bergantian dan sistem secara otomatis menangani situasi seri dengan mengadakan lemparan ulang. Yang paling menonjol dari segi arsitektur perangkat lunak adalah sistem plugin modular yang tidak hanya terbatas pada satu jenis, melainkan mendukung tiga kategori ekstensi: kartu eksperimen kustom, generator papan kustom, dan strategi bot berbasis kecerdasan buatan, yang semuanya dimuat secara dinamis dari file JAR eksternal pada saat runtime. Seluruh fitur ini berjalan di atas framework JavaFX yang memungkinkan pemisahan bersih antara logika presentasi dan logika bisnis melalui penggunaan FXML, CSS, serta controller berbasis anotasi.

2. Kakas GUI: JavaFX

JavaFX adalah framework antarmuka pengguna grafis (GUI) modern yang disediakan oleh Oracle sebagai penerus dari Swing dan AWT dalam ekosistem Java. Berbeda dengan Swing yang berbasis komponen berat (heavyweight) dan memiliki keterbatasan dalam hal styling serta animasi, JavaFX dirancang dari awal dengan arsitektur yang lebih ringan, fleksibel, dan kaya fitur. JavaFX menggunakan pendekatan

scene graph, di mana seluruh elemen UI direpresentasikan sebagai node-node dalam sebuah struktur pohon hirarkis yang terpusat pada satu root node. Pendekatan ini memungkinkan rendering yang efisien karena JavaFX mampu melacak perubahan pada node-node individual dan hanya merender ulang bagian yang terpengaruh, sehingga performa tetap optimal meskipun antarmuka menjadi kompleks. JavaFX juga memiliki dukungan bawaan untuk styling melalui Cascading Style Sheets (CSS), yang memungkinkan pengembang untuk memisahkan sepenuhnya aspek visual dari aspek logika kode, serupa dengan pengembangan web modern. Lebih dari itu, JavaFX menyediakan API untuk efek visual seperti bayangan, pencahayaan, transisi, animasi timeline, dan bahkan pemutaran media audio dan video.

Dalam konteks pengembangan aplikasi Banana Republic, JavaFX menjadi pilihan yang sangat tepat karena memungkinkan pembuatan antarmuka yang kaya secara visual, mulai dari papan hexagonal dengan gambar tekstur tile, animasi gelembung latar belakang lautan, efek drop shadow pada token angka, hingga pemutaran backsound musik dan efek suara, semuanya dapat diimplementasikan menggunakan API native JavaFX tanpa memerlukan pustaka eksternal untuk rendering grafis. Selain itu, kemampuan JavaFX untuk memisahkan definisi UI ke dalam file FXML membuat kode Java menjadi lebih bersih dan mudah dikelola, karena desainer UI dapat bekerja pada file FXML dan XML sementara pengembang backend dapat fokus pada logika controller dan model bisnis. Siklus hidup (lifecycle) aplikasi JavaFX mengikuti pola yang terdefinisi dengan jelas oleh kelas `javafx.application.Application`. Ketika sebuah aplikasi JavaFX dijalankan, method statis `main` memanggil `Application.launch(args)`, yang secara internal mempersiapkan runtime JavaFX dan menginisialisasi toolkit. Proses ini kemudian memicu tiga fase utama: `init()`, `start(Stage)`, dan `stop()`. Fase `init()` dieksekusi pada background thread dan digunakan untuk melakukan inisialisasi berat atau memuat resource yang memerlukan waktu, tanpa memblokir thread utama UI. Fase `start(Stage)` adalah inti dari aplikasi JavaFX, dimana method ini menerima objek `Stage` utama sebagai parameter. `Stage` dalam JavaFX merupakan window atau frame utama aplikasi, yang setara dengan `JFrame` pada Swing. Di dalam method `start()`, pengembang bertanggung jawab untuk membuat `Scene`, yaitu container yang menampung seluruh hierarki node UI, lalu mengaitkannya ke `Stage`, mengatur judul window, ukuran minimal, dan akhirnya memanggil `stage.show()` untuk menampilkan window kepada pengguna. Fase terakhir, `stop()`, dipanggil ketika aplikasi akan ditutup, baik karena pengguna menutup window maupun karena dipanggil secara programmatic, dan fase ini digunakan untuk membersihkan resource atau menyimpan state terakhir.

Dalam implementasi Banana Republic, kelas App meng-extend Application dan method start()-nya menginisialisasi Navigator dengan stage yang diterima, mengatur judul aplikasi menjadi "Banana Republic", menetapkan ukuran minimal window, dan meminta Navigator untuk memuat tampilan utama menu melalui file FXML. Setelah itu, stage.show() dipanggil dan aplikasi mulai berjalan. Method init() dan stop() tidak di-override secara eksplisit dalam aplikasi ini, namun JavaFX secara otomatis tetap menjalankan siklusnya. Penggunaan Navigator sebagai class utilitas menambahkan satu lapisan abstraksi di atas JavaFX lifecycle standar, di mana Navigator membuat satu Scene tunggal yang berisi StackPane sebagai shell, dan semua navigasi antar halaman (dari main menu ke lobby, ke game, ke dialog overlay) dilakukan dengan memanipulasi children dari StackPane tersebut, sehingga tidak perlu membuat Scene atau Stage baru setiap kali berpindah halaman.

Agar sebuah GUI JavaFX dapat muncul dan berfungsi dengan baik, terdapat beberapa komponen utama yang harus tersedia dan saling terkait. Komponen pertama dan yang paling fundamental adalah sebuah kelas yang meng-extend javafx.application.Application. Kelas ini menjadi entry point resmi aplikasi JavaFX dan wajib meng-override method start(Stage) untuk mendefinisikan apa yang akan ditampilkan pada window utama. Tanpa kelas ini, toolkit JavaFX tidak akan terinisialisasi dan tidak ada window yang dapat dibuat. Komponen kedua adalah Stage, yang merupakan top-level container yang mewakili window fisik pada sistem operasi. Stage menyediakan properti untuk judul, ukuran, icon, serta kemampuan untuk membuka window baru (secondary stage) atau dialog modal. Komponen ketiga adalah Scene, yang merupakan container untuk seluruh hierarki node UI dalam satu window. Scene menentukan dimensi awal area gambar dan menjadi tempat di mana root node dari scene graph ditempatkan. Komponen keempat adalah root node itu sendiri, yang dalam kasus Banana Republic berupa StackPane yang dibuat oleh Navigator. StackPane dipilih karena kemampuannya untuk menumpuk node-node secara bertumpuk (stack), yang sangat ideal untuk mengelola navigasi antar halaman dan menampilkan dialog overlay di atas halaman aktif tanpa memerlukan jendela baru. Komponen kelima adalah FXMLLoader, yang merupakan utilitas JavaFX untuk mem-parse file FXML (berbasis XML) dan mengkonversinya menjadi hierarki objek Node secara otomatis. FXMLLoader juga bertanggung jawab untuk meng-instantiate controller class yang terasosiasi dengan file FXML dan meng-inject objek-objek UI yang diberi fx:id ke dalam field-field controller yang dianotasi dengan @FXML. Komponen enam adalah controller class itu sendiri, yang merupakan kelas Java biasa yang berisi logika presentasi dan event handling. Controller ini harus memiliki method initialize() yang akan dipanggil secara otomatis oleh FXMLLoader setelah seluruh injeksi selesai, sehingga pengembang dapat melakukan setup tambahan seperti meng-attach latar belakang animasi, memainkan background, atau mengisi data awal ke dalam

elemen UI. Dalam Banana Republic, setiap tampilan, baik main menu, lobby, game board, maupun berbagai dialog, memiliki controller masing-masing yang terhubung dengan file FXML-nya, membentuk pola desain Model-View-Controller (MVC) yang terstruktur dan terpisah dengan baik.

JavaFX menyediakan beragam komponen siap pakai yang dapat dikelompokkan ke dalam beberapa kategori utama. Kategori pertama adalah layout containers, yang berfungsi untuk mengatur posisi dan ukuran komponen child-nya. JavaFX menyediakan StackPane untuk tumpukan bertingkat, VBox dan HBox untuk penataan vertikal dan horizontal, GridPane untuk tata letak berbasis baris dan kolom, BorderPane untuk penempatan pada lima area utama (top, bottom, left, right, center), FlowPane untuk penataan yang mengalir seperti teks, dan TilePane untuk tata letak berpetak. Kategori kedua adalah controls, yaitu komponen interaktif yang dapat menerima input dari pengguna. Controls yang paling umum di antaranya adalah Button untuk tombol aksi, Label untuk teks statis, TextField dan TextArea untuk input teks, PasswordField untuk input sandi, CheckBox dan RadioButton untuk pilihan boolean, ComboBox dan ChoiceBox untuk dropdown pilihan, Slider untuk input nilai berbasis geser, dan ProgressBar serta ProgressIndicator untuk indikator kemajuan. Selain itu, JavaFX juga menyediakan Alert, Dialog, dan FileChooser untuk menampilkan dialog informasi, konfirmasi, atau membuka jendela pemilihan file sistem. Kategori ketiga adalah shapes dan graphics, yang memungkinkan penggambaran bentuk geometris dan grafik kustom secara langsung menggunakan kode Java. Shapes yang tersedia meliputi Rectangle, Circle, Ellipse, Polygon, Line, Polyline, dan CubicCurve. Efek visual juga dapat ditambahkan melalui class Effect dan subclass-nya seperti DropShadow, InnerShadow, GaussianBlur, Glow, dan Reflection.

Dalam implementasi HexBoard pada Banana Republic, seluruh papan hexagonal, pipa, bangunan, pelabuhan, dan token angka digambar secara programmatic menggunakan kombinasi Polygon, Line, Circle, Rectangle, dan Text, lengkap dengan efek DropShadow dan ImagePattern untuk memberikan tekstur gambar pada tile-tile. Kategori keempat adalah media, yang mencakup dukungan untuk memutar audio dan video melalui class Media, MediaPlayer, dan MediaView. Dalam Banana Republic, fitur audio diimplementasikan menggunakan MediaPlayer untuk BGM yang berputar secara berulang atau bergantian antar trek, serta untuk memainkan efek suara (SFX) dalam bentuk file WAV dan MP3. Kategori kelima adalah animation dan transitions, yang menyediakan API untuk membuat animasi property, transisi fade, translate, scale, rotate, dan timeline berbasis keyframe. Aplikasi Banana Republic memanfaatkan Timeline dari kategori ini untuk membuat animasi guling dadu yang terdiri dari sepuluh frame kecepatan berbeda sebelum menampilkan angka final. Terakhir, kategori keenam adalah

web dan text, yang termasuk WebView untuk menampilkan konten HTML5, HTMLEditor untuk rich text editing, dan class-class styling seperti Font, Text, dan TextFlow untuk manipulasi teks yang kompleks. Seluruh komponen ini dapat di-styling menggunakan CSS eksternal maupun inline, memungkinkan tampilan yang sepenuhnya dapat dikustomisasi sesuai kebutuhan desain aplikasi.

3. Plugin & Class Loader

Class loader dalam ekosistem Java merupakan mekanisme fundamental yang bertanggung jawab atas pemuatan kelas-kelas Java ke dalam Java Virtual Machine (JVM) secara dinamis. Secara default, Java menggunakan hierarki class loader yang terdiri dari Bootstrap ClassLoader (untuk kelas-kelas inti JDK), Extension ClassLoader (untuk ekstensi platform), dan Application ClassLoader (untuk kelas-kelas yang terdapat dalam classpath aplikasi). Namun, kemampuan sebenarnya dari mekanisme class loading yang menarik bagi pengembang aplikasi yang memerlukan modularitas adalah kemungkinan untuk membuat class loader kustom (custom class loader) yang dapat memuat kelas dari sumber-sumber di luar classpath standar, seperti file JAR eksternal, direktori, atau bahkan data mentah dari jaringan.

3.1. Class Loader

Dalam aplikasi Banana Republic, mekanisme ini diimplementasikan menggunakan URLClassLoader, yang merupakan subclass dari `java.net.URLClassLoader` dan memungkinkan JVM untuk memperlakukan satu atau lebih URL (biasanya merujuk ke file JAR atau direktori) sebagai classpath tambahan pada saat runtime. Proses kerjanya dimulai dengan pembuatan objek URL yang merujuk ke file JAR plugin yang dipilih oleh pengguna melalui FileChooser. Kemudian, sebuah instance URLClassLoader dibuat dengan meneruskan array URL tersebut sebagai classpath baru dan menggunakan class loader saat ini (biasanya Application ClassLoader) sebagai parent class loader. Pemilihan parent class loader yang tepat sangat penting karena memastikan bahwa kelas-kelas yang sudah tersedia dalam aplikasi utama, seperti interface plugin, model, dan exception, dapat diakses oleh kelas-kelas yang dimuat dari JAR plugin tanpa memerlukan duplikasi dependency.

Setelah class loader terinisialisasi, langkah selanjutnya adalah menelusuri seluruh entries di dalam file JAR menggunakan class JarFile. Setiap entry yang berakhiran .class diekstrak, path-nya dikonversi ke fully-qualified class name dengan mengganti slash (/) menjadi titik (.), dan kemudian kelas tersebut dimuat ke memori menggunakan method `Class.forName(className, true, classLoader)` dengan menyertakan class loader kustom sebagai parameter. Pemuatan kelas ini bukan sekadar membaca bytecode dari disk; JVM juga mengeksekusi static initializer kelas, memverifikasi struktur bytecode, dan menyelesaikan (resolve) seluruh referensi ke kelas-kelas lain yang diperlukan. Setelah kelas berhasil dimuat, aplikasi memeriksa apakah kelas tersebut mengimplementasikan interface plugin yang diharapkan menggunakan method `isAssignableFrom()` dari objek `Class` interface. Jika kelas lolos verifikasi dan bukan merupakan interface itu sendiri, maka objek dari kelas tersebut dibuat melalui mekanisme Java Reflection, yaitu dengan memanggil `getDeclaredConstructor()` untuk mendapatkan constructor default (tanpa parameter) dan kemudian `newInstance()` untuk meng-instantiate objek. Objek yang telah terbentuk ini kemudian di-cast ke tipe interface plugin dan siap digunakan. Setelah proses pemuatan selesai, aplikasi secara eksplisit menutup `URLClassLoader` untuk melepaskan handle file JAR dan mencegah kebocoran resource pada sistem operasi. Seluruh proses ini dilindungi oleh blok try-catch yang komprehensif untuk menangani berbagai kemungkinan kegagalan, mulai dari file JAR yang tidak ditemukan, kelas yang tidak mengimplementasikan interface yang tepat, hingga kegagalan instansiasi akibat constructor yang tidak tersedia atau dependency yang hilang.

3.2. Plugin

Fitur plugin dalam aplikasi Banana Republic tidak terbatas pada satu jenis ekstensi saja, melainkan telah berkembang menjadi tiga kategori plugin yang masing-masing melayani kebutuhan fungsionalitas yang berbeda, namun semuanya dibangun di atas fondasi mekanisme `ClassLoader` yang sama.

■ Card Plugin

Kategori pertama adalah Experiment Card Plugin, yang merupakan sistem plugin paling awal dan paling inti. Sistem ini berbasis interface `ExperimentCard` yang mendefinisikan kontrak minimal berupa tiga method: `getCardName()` untuk mengembalikan nama kartu, `getDescription()` untuk mengembalikan deskripsi efek kartu, dan `applyEffect()` untuk mengeksekusi logika efek kartu ketika dimainkan. Plugin loader untuk kategori ini, bernama `PluginLoader`, bertugas menelusuri file JAR yang dipilih pengguna, mencari seluruh kelas yang mengimplementasikan interface `ExperimentCard`, menginstansiasi masing-masing kelas tersebut, dan membungkusnya menggunakan `PluginExperimentCardAdapter`. Adapter ini merupakan class bridge yang meng-extend `DevelopmentCard` (tipe kartu internal game) dan mendelegasikan pemanggilan method `play()` ke method `applyEffect()` dari objek plugin, sehingga kartu-kartu yang berasal dari plugin eksternal dapat disatukan (shuffled) ke dalam deck kartu pengembangan (development deck) dan dimainkan seolah-olah mereka adalah kartu internal asli. Sebagai langkah keamanan dan isolasi, sistem ini menggunakan `PluginApiViews` untuk menyediakan wrapper read-only (view objects) seperti `GameStateView`, `PlayerView`, dan `BankView` yang hanya mengekspos method-method aman ke plugin tanpa memberikan akses langsung ke objek mutable internal engine, sehingga plugin tidak dapat merusak state game secara tidak sengaja atau sengaja.

■ Map Plugin

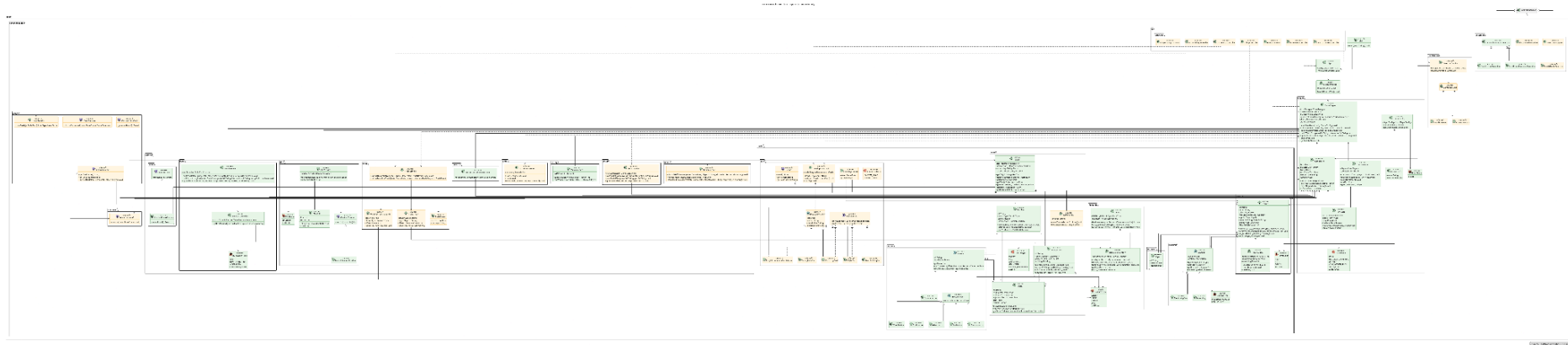
Kategori kedua adalah Map Plugin, yang diimplementasikan melalui interface `MapGeneratorPlugin`. Interface ini memungkinkan pengembang pihak ketiga untuk membuat generator papan permainan kustom dengan aturan peletakan tile, token, dan pelabuhan yang berbeda dari standar. `MapPluginLoader` bekerja dengan pola yang identik: memuat JAR, mencari implementor `MapGeneratorPlugin`, menginstansiasi, dan mengembalikan objek generator yang kemudian digunakan oleh `GameEngine` untuk membangun papan saat permainan baru dimulai. Jika tidak ada plugin peta yang dipilih, aplikasi secara otomatis fallback ke `StandardMapGenerator` yang merupakan implementasi bawaan.

■ Bot Plugin

Kategori ketiga adalah Bot Plugin, yang diwakili oleh interface `PlayerStrategy`. Plugin ini memungkinkan pembuatan kecerdasan buatan (AI) yang dapat mengontrol satu atau lebih pemain dalam permainan secara otomatis. `BotPluginLoader` memuat JAR yang berisi strategi bot, mencari kelas yang mengimplementasikan `PlayerStrategy`, dan menginstansiasinya. Strategi bot ini kemudian diikat ke pemain tertentu dalam lobby, dan selama permainan `GameEngine` akan secara berkala memanggil method pada `PlayerStrategy` untuk menentukan aksi apa yang akan dilakukan bot pada gilirannya, mulai dari fase setup penempatan post dan pipe, pelemparan dadu, penanganan Nim on Ungu, hingga aksi perdagangan dan pembangunan. Seluruh aksi bot dijalankan melalui API `BotActions` yang menyediakan method-method konkret seperti `buildPost()`, `buildPipe()`, `trade()`, dan `endTurn()`, sehingga pengembang plugin bot tidak perlu mengakses langsung ke objek internal game.

Ketiga sistem plugin ini diintegrasikan ke dalam antarmuka pengguna melalui komponen `FileChooser` pada JavaFX. Di `LobbyController`, pengguna dapat menelusuri dan memilih file JAR untuk map plugin dan bot plugin sebelum memulai permainan baru, sementara di `SettingsDialogController` pengguna dapat memuat plugin kartu eksperimen tambahan kapan saja selama permainan berlangsung. Dengan mendesain sistem plugin yang seragam dan modular ini, aplikasi Banana Republic tidak hanya menjadi board game yang lengkap secara fungsional, tetapi juga menjadi platform yang terbuka untuk ekspansi dan kustomisasi tanpa batas, di mana komunitas pengembang dapat menciptakan pengalaman bermain yang selalu baru melalui kartu, peta, dan lawan berbasis AI yang mereka ciptakan sendiri.

4. Class Diagram



Class diagram Banana Republic terbagi menjadi layer model, service, engine, controller, persistence, dan plugin. GameEngine sebagai pusat bertugas mengorkestrasi jalannya permainan, dimana GameState disimpan dan service-service khusus dipakai seperti BuildService, DiceService, TradeService, ResourceProductionService, NimonsService, VictoryService, SetupService, dan TurnTimerService. Dengan pembagian ini, controller JavaFX tidak langsung mengubah model secara bebas, tetapi memanggil method pada GameEngine lalu meneruskan pekerjaan ke tiap service.

Pada bagian model, GameState menyimpan informasi Board, daftar Player, Bank, TurnState, posisi Nimons, deck kartu, pemenang, serta pemegang kartu spesial. Board sendiri berisi kumpulan HexTile, Intersection, Path, dan Harbor. HexTile mewakili petak penghasil resource, Intersection menjadi tempat bangunan, sedangkan Path menjadi tempat pipa dan pelabuhan. Pemain memiliki ResourceInventory, PlayerSupply, daftar bangunan, daftar pipa, dan kartu yang sedang dipegang. Relasi ini membuat state permainan bisa ditelusuri dari level paling besar, yaitu GameState, sampai detail papan dan kepemilikan pemain.

Diagram memanfaatkan inheritance dan interface. Bangunan diturunkan dari abstract class Building, lalu diimplementasikan oleh MonitoringPost dan Laboratory. Pelabuhan diturunkan dari abstract class Harbor, dengan GenericHarbor dan beberapa SpecificHarbor seperti

WoodHarbor, BrickHarbor, WheatHarbor, OreHarbor, dan BananaHarbor. Untuk kartu, DevelopmentCard menjadi dasar bagi kartu seperti KnightCard, RoadBuildingCard, MonopolyCard, VictoryPointCard, dan PluginExperimentCardAdapter. Bagian plugin juga terhubung lewat PluginLoader, yang memuat class dari JAR eksternal dan membungkusnya dengan adapter agar tetap bisa diperlakukan sebagai kartu domain biasa.

5. Konsep OOP

5.1. Inheritance

- MonitoringPost (src/main/java/com/bananarepublic/model/building/MonitoringPost.java) dan Laboratory (src/main/java/com/bananarepublic/model/building/Laboratory.java) merupakan anak dari abstract class Building (src/main/java/com/bananarepublic/model/building/Building.java).
- GenericHarbor (src/main/java/com/bananarepublic/model/harbor/GenericHarbor.java) merupakan anak dari abstract class Harbor (src/main/java/com/bananarepublic/model/harbor/Harbor.java).
- WoodHarbor, BrickHarbor, WheatHarbor, OreHarbor, dan BananaHarbor merupakan anak dari SpecificHarbor, sedangkan SpecificHarbor merupakan anak dari Harbor.
- KnightCard, RoadBuildingCard, MonopolyCard, VictoryPointCard, dan PluginExperimentCardAdapter merupakan anak dari abstract class DevelopmentCard (src/main/java/com/bananarepublic/model/card/DevelopmentCard.java).
- InvalidMoveException, InvalidTradeException, dan InsufficientResourceException merupakan anak dari GameRuleViolationException (src/main/java/com/bananarepublic/exception/GameRuleViolationException.java), GameRuleViolationException merupakan anak dari RuntimeException.

5.2. Composition

- `GameEngine` (`src/main/java/com/bananarepublic/engine/GameEngine.java`) memiliki `GameState`, `TurnManager`, dan banyak kelas `Service` seperti `DiceService`, `BuildService`, `TradeService`, `NimonService`, `VictoryService`, `TurnTimerService`, dan `SaveLoadService`.
- `GameState` (`src/main/java/com/bananarepublic/engine/GameState.java`) memiliki `Board`, `List<Player>`, `Bank`, `TurnState`, dan `DevelopmentDeck`.
- `Board` (`src/main/java/com/bananarepublic/model/board/Board.java`) memiliki `Map<String, HexTile>`, `Map<String, Intersection>`, `Map<String, Path>`, dan `Map<String, Harbor>`.
- `Player` (`src/main/java/com/bananarepublic/model/player/Player.java`) memiliki `ResourceInventory`, `PlayerSupply`, `List<Building>`, `List<Pipe>`, `Set<SpecialCardType>`, dan `List<DevelopmentCard>`.
- `Intersection` memiliki satu `Building` opsional, sedangkan `Path` memiliki satu `Pipe` opsional dan satu `Harbor` opsional.

5.3. Interface

- `SaveLoadService` (`src/main/java/com/bananarepublic/persistence/SaveLoadService.java`) mendefinisikan kontrak `save` dan `load`, lalu diimplementasikan oleh `SnapshotSaveLoadService`.
- `BoardFactory` (`src/main/java/com/bananarepublic/service/board/BoardFactory.java`) mendefinisikan kontrak pembuatan `Board`, lalu diimplementasikan oleh `StandardBoardFactory`.
- `RandomProvider` (`src/main/java/com/bananarepublic/service/dice/RandomProvider.java`) digunakan oleh `DiceService`, lalu diimplementasikan oleh `DefaultRandomProvider`.
- `Card` dan `PlayableCard` (`src/main/java/com/bananarepublic/model/card/Card.java`, `PlayableCard.java`) menjadi kontrak untuk kartu yang punya identitas dan dapat dimainkan.
- Interface plugin seperti `ExperimentCard`, `GameState`, `Player`, dan `Bank` pada package `src/main/java/com/bananarepublic/plugin/` menjadi API terbatas untuk plugin eksternal.

5.4. Method Overriding dan Method Overloading

- `MonitoringPost.getType()` dan `MonitoringPost.canBeUpgraded()` meng-override abstract method dari `Building`.
- `Laboratory.getType()` dan `Laboratory.canBeUpgraded()` meng-override abstract method dari `Building`.
- `SpecificHarbor.getRatio()` dan `SpecificHarbor.canTradeWith()` meng-override method dari `Harbor`.
- `GameEngine.playDevelopmentCard()` dioverload menjadi beberapa versi: tanpa parameter tambahan, dengan `ResourceType`, dengan `List<String>`, serta dengan `tileId` dan `victimPlayerId`.
- `GameEngine.buildRoad()` dan `GameEngine.buildWatchPost()` dioverload untuk membedakan pembangunan normal dan pembangunan saat setup.

5.5. Polymorphism

- `BuildService.upgradeLaboratory()` mengambil `Building` existing, tetapi object yang digunakan dapat berupa `MonitoringPost`. Setelah upgrade object dapat diganti menjadi `Laboratory`.
- `ResourceProductionService` memanggil `building.getProductionAmount()` terhadap tipe `Building`, sehingga produksi bekerja untuk `MonitoringPost` maupun `Laboratory`.
- `TradeService.resolveBestRatio()` memakai tipe `Harbor`, tetapi object dapat berupa `GenericHarbor`, `WoodHarbor`, `BrickHarbor`, dan harbor spesifik lainnya.
- `GameEngine.playDevelopmentCard()` memakai tipe `DevelopmentCard`, tetapi method `play()` yang berjalan mengikuti subclass aktual seperti `KnightCard`, `MonopolyCard`, `RoadBuildingCard`, atau `PluginExperimentCardAdapter`.
- `GameEngine` menyimpan `SaveLoadService`, tetapi implementasi aktualnya adalah `SnapshotSaveLoadService`.

5.6. Java API Collection

- Board menggunakan Map<String, HexTile>, Map<String, Intersection>, Map<String, Path>, dan Map<String, Harbor> untuk akses cepat berdasarkan id.
- Player menggunakan List<Building>, List<Pipe>, List<DevelopmentCard>, dan EnumSet<SpecialCardType> untuk menyimpan kepemilikan pemain.
- ResourceInventory menggunakan EnumMap<ResourceType, Integer> agar penyimpanan resource efisien dan aman berdasarkan enum.
- GameEngine banyak memakai Stream API, misalnya untuk mencari id valid pembangunan, target Nimon, dan daftar pemain yang harus discard.
- VictoryService memakai HashSet<Path> pada DFS longest road agar path yang sama tidak dihitung dua kali.

5.7. SOLID

■ S

- BuildService hanya mengurus validasi dan eksekusi pembangunan, sedangkan harga bangunan dipisahkan ke BuildCostProvider.
- TradeService hanya menangani domestic trade dan maritime trade, bukan produksi resource atau penentuan kemenangan.
- ResourceProductionService hanya menangani produksi resource berdasarkan hasil dadu dan kondisi papan.

■ O

- Sistem kartu terbuka untuk kartu baru dengan menambah subclass DevelopmentCard tanpa mengubah struktur utama Player atau DevelopmentDeck.
- Sistem harbor terbuka untuk jenis harbor baru dengan menambah subclass Harbor atau SpecificHarbor.
- Sistem save/load dapat diperluas dengan implementasi baru dari SaveLoadService tanpa mengubah kontrak pemakaian di GameEngine.

■ L

- MonitoringPost dan Laboratory dapat dipakai sebagai Building pada Intersection, Player, dan ResourceProductionService.
- Semua subclass Harbor dapat dipakai sebagai Harbor di Path dan TradeService.
- Semua subclass DevelopmentCard dapat disimpan dalam DevelopmentDeck dan dimainkan melalui referensi DevelopmentCard.

■ I

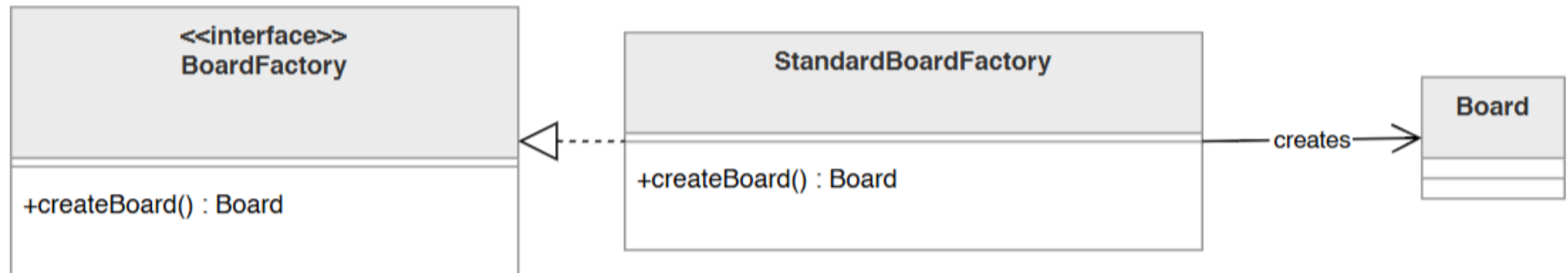
- RandomProvider kecil dan spesifik, hanya berisi nextInt(int bound).
- BoardFactory hanya berisi createBoard(), sehingga implementor tidak dipaksa menangani hal lain.
- Interface plugin GameState, Player, dan Bank hanya membuka operasi yang perlu untuk plugin, bukan seluruh class internal game.

■ D

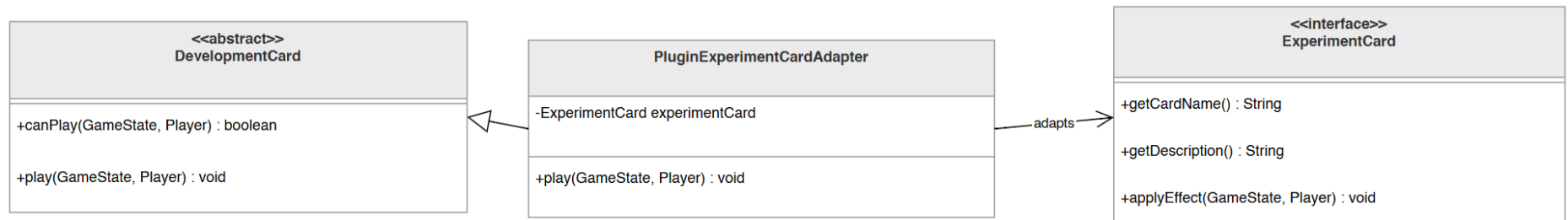
- DiceService bergantung pada interface RandomProvider, bukan langsung pada Random.
- GameEngine menerima SaveLoadService lewat constructor, sehingga tidak wajib terikat ke satu implementasi save/load.
- Plugin berinteraksi lewat interface ExperimentCard, GameState, Player, dan Bank, bukan langsung ke semua class internal domain.

5.8. Design Pattern

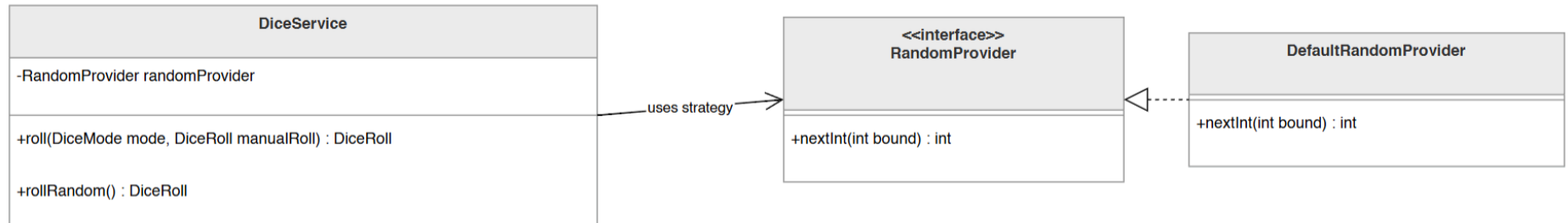
- Factory Pattern: BoardFactory sebagai interface factory dan StandardBoardFactory sebagai concrete factory untuk membuat Board. Keuntungannya, proses pembuatan papan yang kompleks tidak tersebar di GameEngine. Jika nanti ada papan random atau papan dari plugin, cukup buat factory baru yang menghasilkan Board.



- Adapter Pattern: `PluginExperimentCardAdapter` membungkus object plugin yang mengimplementasikan `ExperimentCard` agar bisa diperlakukan sebagai `DevelopmentCard`. Tanpa adapter ini, kartu plugin tidak bisa langsung masuk ke `DevelopmentDeck` dan flow kartu utama.



- Strategy Pattern: `DiceService` memakai `RandomProvider` sebagai strategi sumber angka acak. Implementasi default-nya adalah `DefaultRandomProvider`, tetapi untuk testing bisa diganti provider lain yang deterministik.



5.9. Reflection

- `PluginLoader.instantiateCard()` (`src/main/java/com/bananarepublic/plugin/PluginLoader.java`) menggunakan `Class.forName(className, true, classLoader)` untuk memuat class plugin dari JAR eksternal saat runtime.
- Method yang sama memakai `clazz.getDeclaredConstructor().newInstance()` untuk membuat object plugin tanpa mengetahui class konkretnya saat compile time.
- `PluginLoader.findExperimentCardClasses()` membaca isi JAR dengan `JarFile`, lalu mencoba memuat setiap `.class` dan memeriksa apakah class tersebut mengimplementasikan `ExperimentCard`.
- `PluginLoader.loadSingleCardFromJar()` memakai mekanisme yang sama untuk memuat kembali kartu plugin ketika save file di-load.
- Validasi `ExperimentCard.class.isAssignableFrom(clazz)` digunakan agar hanya class yang sesuai kontrak plugin yang diterima.

5.10. Multithreading

- `TurnTimerService` (`src/main/java/com/bananarepublic/service/timer/TurnTimerService.java`) memakai `ScheduledExecutorService` untuk menjalankan timer giliran pada thread terpisah bernama `turn-timer`.
- `TurnTimerService.start(...)`, `stop()`, `getRemainingSeconds()`, dan `close()` memakai `synchronized` agar akses ke `remainingSeconds`, `activeTask`, dan callback timer aman dari race condition.

- Platform.runLater(...) digunakan di beberapa controller dan AudioEngine untuk memastikan update UI JavaFX tetap dijalankan pada JavaFX Application Thread.
- GameController memakai Timeline untuk tick UI dan animasi dadu, sehingga animasi berjalan berbasis event JavaFX tanpa memblokir input user.
- Thread timer dibuat sebagai daemon thread, sehingga tidak menahan aplikasi ketika window utama sudah ditutup.

5.11. Assertion

- GameSaveData.fromState() memakai assertion untuk memastikan jumlah player yang disimpan sama dengan jumlah player pada GameState.
- GameSaveData.toGameState() memakai assertion untuk memastikan jumlah tile pada board hasil load sama dengan jumlah tile pada data save.
- GameSaveData.restorePlayers() memakai assertion untuk memastikan jumlah player hasil restore sama dengan jumlah player pada save data.
- DeckSaveData.toDeck() memakai assertion untuk memastikan ukuran draw pile deck hasil restore sesuai dengan data yang disimpan.
- TurnState.restore() memakai assertion untuk memastikan jumlah newlyBoughtCardIds dan pendingDiscardPlayerIds setelah restore sama dengan data asal.

6. Bonus Yang dikerjakan

6.1. Generator Peta Baru

Pendekatan:

- Membuat interface MapGeneratorPlugin dengan method Board generateBoard()
- Membuat MapPluginLoader yang menggunakan URLClassLoader untuk memuat JAR eksternal secara dinamis

- JAR dipindai untuk menemukan kelas yang mengimplementasikan MapGeneratorPlugin
- Objek plugin di-instantiate via Java Reflection dan digunakan sebagai pengganti StandardMapGenerator bawaan
- Integrasi UI melalui tombol "Browse Map Plugin" di LobbyController sebelum permainan dimulai
- File kunci: MapGeneratorPlugin.java, MapPluginLoader.java, StandardMapGenerator.java, LobbyController.java

6.2. Pemain Bot

Pendekatan:

- Membuat interface PlayerStrategy dengan method Action takeTurn(GameState state)
- Membuat BotPluginLoader dengan pola identik menggunakan URLClassLoader
- Membuat Action sebagai interface kontrak aksi bot dan BotActions sebagai API konkret
- Saat giliran bot, GameEngine memanggil takeTurn() dari strategi yang dimuat
- Integrasi UI melalui toggle BOT per pemain dan tombol browse di LobbyController
- File kunci: PlayerStrategy.java, BotPluginLoader.java, BotActions.java, Action.java, LobbyController.java

6.3. BGM & SFX Engine (Audio Player)

Pendekatan:

- Membuat singleton AudioEngine yang mengelola pemutaran audio via JavaFX MediaPlayer
- BGM terdiri dari 3 file MP3: main_menu.mp3 (loop), game1.mp3 dan game2.mp3 (berotasi otomatis saat track selesai)
- SFX terdiri dari 8 file audio (WAV/MP3): click, error, build, dice, trade, achievement, dagger, nimon_ungu
- Semua pemutaran berjalan asynchronous menggunakan Platform.runLater() agar tidak memblokir UI thread
- Settings dialog dilengkapi slider volume BGM dan SFX
- File kunci: AudioEngine.java, resource audio di src/main/resources/audio/

6.4. Animasi Background

Pendekatan:

- Membuat class utilitas LivingBackground dengan 3 variant: PARCHMENT, OCEAN, SLATE
- Menggunakan JavaFX Timeline dan KeyFrame untuk animasi ringan (bukan thread berat)
- Elemen animasi: awan bergerak horizontal, burung camar terbang, kapal mini bobbing
- Semua animasi di-track dalam list CopyOnWriteArrayList agar bisa di-pause/resume global
- File kunci: LivingBackground.java

7. Pembagian Tugas

NIM	NAMA	TUGAS
13524026	Made Branenda Jordhy	- UI JavaFX - Save - Load - Laporan
13524028	Muhammad Nur Majiid	- Card - Plugin - Laporan
13524034	Jason Edward Salim	- Trade System - Nimon - Laporan

13524067	Bryan Pratama Putra Hendra	- UI JavaFX - Save - Load - Laporan
13524068	Athilla Zaidan Zidna Fann	- Setup phase - Turn timer - Victory rules - Laporan